1. TITLE PAGE

Department:Department of Computer Engineering
Course:Concepts of Programming Languages
Project Topic: Term Project: Design and Implement Your Own Turkish-İnspired Programming Language Objective
Implementation Language:Java
Due Date:June 1, 2026

2. GROUP MEMBER

This project was jointly carried out by a 4-person working group, whose details are provided below:

|First Name Last Name | Student ID |

|Semi Kazar|230316066|

| Ufuk Akkuzu|230316049 |

| Berat Uzdil| 230316043 |

| Berkay Altunbağ| 230316009 |

3. INTRODUCTION

The design and implementation of programming languages (compilers and interpreters) is one of the most fundamental theoretical and practical areas of computer science. For source code to be processed by a computer, it must first undergo a series of front-end analysis stages. These analysis stages are, in order:

1.Lexical Analysis:This involves scanning the character stream known as source code, dividing it into meaningful word units (lexemes), and converting these units into Token objects. The component that performs this step is called a Lexical Analyzer(Lexer).

2.Syntax Analysis (Parsing): This is the process of verifying that the generated token stream conforms to the language's formal grammar rules. The component that performs syntax analysis is called a parser . The parser ensures that the code is ready for semantic analysis by verifying the hierarchical relationships between tokens.
3.Symbol Table Management:A data structure that stores user-defined identifiers—such as variables, constants, and functions—defined during the compilation process, providing fast access and type checking.

The primary objective of this project is to design a consistent, original, and flexible programming language with a Turkish vocabulary and syntactic structure, and to implement a compiler front-end for this language based on the design patterns in the 10th edition of Sebesta. In this project, the lexer state machine designs, EBNF grammar rules, and LL(1) recursive-descent parsing approaches learned in theory have been translated into practical code.

4. BASE TEXTBOOK CODE USED

In this project, the sample code provided in Robert W. Sebesta, Concepts of Programming Languages, 10th Edition, Chapter 4 as recommended for the course was used as the base code.

The structural algorithm designed in C in the book has been adapted and extended to the Java language to align with today's modern object-oriented programming practices. The structural relationships between the book's code and the TürkDil project, along with the changes made, are summarized below:

A. Core Structures Preserved in the Book (What is Kept)

Character Classification Logic: The method of reading and classifying characters one by one (letters, numbers, etc.) and skipping spaces has been preserved.

lex() Flow: The behavior of the `lex()` function on page 172 of the book—specifically, the logic of reading the next character, determining the token type, and preparing for the next step—has been preserved exactly as described.

Parser Function Hierarchy: The recursive structure of the `expr()`, `term()`, and `factor()` functions used to parse arithmetic expressions, as well as the operator precedence order, has been preserved.

B. Modified Structures (What is Modified)

Transition from C to Java: The C-based pointer and global variable-based state management in the book has been converted to object-oriented, encapsulated, and safe class structures in Java. The `Lexer` and `Parser` states are stored in object fields.

Dynamic Lexical Analysis: The statically sized character array (`char lexeme[100]`) in the book has been replaced with Java's dynamically sized `StringBuilder` structure, thereby eliminating the risk of buffer overflows.

Token Representation: The integer-based token codes in the book (`int nextToken`) have been converted into a custom object named `Token` and a descriptive Java `enum` structure named `TokenType`. Each token now also carries the line number where it appears in the source code (`line`). This makes error reporting extremely precise.

C. New Features (What's New)

Turkish Character Support: Instead of the ASCII-based `isalpha()` check in the book, a custom `isLetter()` check has been added that supports the uppercase and lowercase Turkish characters `ç, ğ, ı, ö, ş, ü, Ç, Ğ, İ, Ö, Ş, Ü`.

Symbol Table: A `SymbolTable.java` class, based on Java's `LinkedHashMap`, has been added to distinguish between keywords and identifiers that are not found in the book's code.

Extended Syntax Rules: The parser in the book, which parses only simple arithmetic expressions (`expr`), has been fully expanded to include variable declarations (`variable`), conditional structures (`if`/`else`), `while` loops (`loop`), `for` loops (`for`), increment/decrement (`++`/`--`), and print statements (`print`) statements.

5. OVERVIEW OF THE DESIGNED LANGUAGE

It is a Turkish-inspired scripting language developed to help Turkish-speaking aspiring software developers and students more easily grasp the underlying logic of programming languages. Simplicity, consistency, and readability were prioritized in the language's design.

Basic Features of the Language and Design Decisions:

Variable Declaration: All variables are declared using the `variable` keyword without specifying a type. When declaring a variable, an initial value can be assigned (`variable x = 10;`) or it can be left unassigned (`variable result;`).

Statement Terminator: All statements are terminated with a `;` (semicolon), just as in C, C++, and Java.

Block Structure: Condition and loop bodies consist of code blocks delimited by `{ }` (curly braces).

Increment and Decrement Operators: The `++` and `--` operators are supported for incrementing and decrementing variable values.

Printing to the Screen: The `yazdir(...)` function, which is built into the language, is used for output operations.

6. RESERVED KEYWORDS AND TOKENS

The reserved words, operators, and separators defined in the programming language are listed in the table below, along with their corresponding token types:

| :--- | :--- | :--- |
DEGISKEN	`degisken`	Variable declaration keyword (var / let)
EGER	`eger`	Start of conditional structure (if)
YOKSA	`yoksa`	Negative condition (else)
DONGU	`dongu`	Conditional loop structure (while)
ICIN	`icin`	Iterative loop structure (for)
YAZDIR	`yazdir`	Command to output to the screen (print)
DOGRU	`dogru`	Logical true constant (true)
YANLIS	`yanlis`	Logical false constant (false)
VE	`ve`		Logical AND operator (&&)
VEYA	`veya`	Logical OR operator (||)
DEGIL	`degil`	Logical NOT operator (!)
INT_LITERAL	Number literals (e.g., `42`)	Integer constants in decimal base
IDENTIFIER	Kullanıcı tanımlı (örn: `sayac`)	User-defined (e.g., `counter`)	Variable or function names (supports Turkish characters)
PLUS	`+`	Multiplication operator
MINUS	`-`	Subtraction or unary minus operator
MULT	`*`	Multiplication operator
DIV	`/`	Division operator
ASSIGN	`=`	Assignment operator
INCREMENT	`++`	Increment operator (increases value by 1)

DECREMENT	`--`	Decrement operator (decreases value by 1)
EQ	`==`	Equality comparison operator
NEQ	`!=`	Inequality comparison operator
LT	`<` Less than relational operator
GT	`>`	Greater than relational operator
LTE	`<=`	Less than or equal to relational operator
GTE	`>=`	Greater than or equal to relational operator
LPAREN / RPAREN	`(` / `)`	Parentheses for parameters and grouping
LBRACE / RBRACE	`{` / `}`	Curly braces for code block delimiters
SEMICOLON	`;`	Statement terminator semicolon
EOF	`EOF`	End-of-file (EOF) character
UNKNOWN	Undefined Characters	Invalid characters with no equivalent in the language

7. BNF / EBNF GRAMMAR

The syntax rules of the language are formally defined below in Extended Backus-Naur Form (EBNF), which directly guides the recursive-descent logic implemented in the parser class:

```
ebnf
( Program start and sequence of expressions )
<program>          ::= <statement>*

(* Temel Deyim Türleri *)
<statement>        ::= <declaration>
                     | <assignment>
                     | <condition>
                     | <loop>
                     | <forLoop>
                     | <print>
(Variable Declaration)
<declaration>      ::= "degisken" IDENTIFIER [ "=" <expr> ] ";"
(Assignment and Value Modification )
<assignment>       ::= IDENTIFIER "=" <expr> ";"
                     | IDENTIFIER "++" ";"
                     | IDENTIFIER "--" ";"
(Assignment Structure Within a Loop (Without Semicolons))
<assignmentNoSemi>  ::= IDENTIFIER "=" <expr>
                                        | IDENTIFIER "++"
                                        IDENTIFIER "--"

(Conditional Structure (Eger-Yoksa))
<condition>        ::= "eger" "(" <boolExpr> ")" "{" <statement>* "}" [ "yoksa" "{" <statement>* "}" ]
```

(Loop Structure (While))

<loop> ::= "dongu" "(" <boolExpr> ")" "{" <statement>* "}"

(Icin Structure (For))

<forLoop> ::= "icin" "(" <assignmentNoSemi> ";" <boolExpr> ";" <assignmentNoSemi> ")" "{" <statement>* "}"

(Print to Screen)

<print> ::= "yazdir" "(" <expr> ")" ";"

(Logical / Comparative Expression)

<boolExpr> ::= "dogru"

| "yanlis"

| <expr> <relOp> <expr>

Comparison Operators
<relOp> ::= "==" | "!=" | "<" | ">" | "<=" | ">="

(Arithmetic Expressions (Order of Operations))

<expr> ::= <term> { ("+" | "-") <term> }

<term> ::= <factor> { ("*" | "/") <factor> }

<factor> ::= "(" <expr> ")"

| "-" <factor>

| INT_LITERAL

| IDENTIFIER

8. SÖZCÜKSEL ÇÖZÜMLEME TASARIMI / LEXICAL ANALYZER DESIGN

The lexical analyzer (`Lexer.java`) generates tokens by scanning the source text character by character. Our design consists of the following stages:

1. Skipping Whitespace and Comments (`skipWhitespaceAndComments`):
Whitespace, tabs, leading spaces, and newline characters (`\n`, which increments the line counter) are skipped.
Single-line comments starting with `//` are scanned up to the end of the line and skipped.
Multi-line block comments enclosed between `/*` and `*/` are completely ignored.

2. Character Classification and Turkish Support: The `isLetter(char c)` method is used to verify whether a character is a letter. In addition to the standard `Character.isLetter(c)` check, this method also checks the Turkish character set (`çğışüöÇĞİŞÜÖ`).

3. Tokenization Flow: Distinguishing Between Tokens and Keywords:When a letter is encountered, characters are accumulated as long as the sequence continues with a letter, digit, or underscore (`_`). The resulting word is looked up in the Symbol Table. If it is registered as a keyword in the Symbol Table, the corresponding keyword token (`VARIABLE`, `IF`, etc.) is returned. If it is not registered, a new `IDENTIFIER` is accepted.
Numeric Literals: When a number is encountered, consecutive digits are scanned to generate an `INT_LITERAL`.
Binary Operators:When the characters `+`, `-`, `=`, `<`, `>`, or `!` are read, the next character is checked (`peekNext`) to identify binary operators (`++`, `--`, `==`, `<=`, `>=`, `!=`). If an `=` does not follow the `!` character, a lexical error (`LexerException`) is thrown because the exclamation mark is not defined as a standalone character in the language.

9. SYNTAX ANALYZER DESIGN

The syntax parser (`Parser.java`) uses a recursive-descent (LL(1)) architecture. Each EBNF rule corresponds to a method within the parser class.
Parser Operating Principles:
LL(1) Approach: The parser scans from left to right (L), generates the leftmost derivation (L), and looks ahead by 1 token to make a decision (Lookahead = 1).
Token Consumption (Consume): The `consume(TokenType type, String context)` method checks whether the current active token type matches the expected type. If they match, it consumes the token and advances the token position by 1. If they do not match, it throws a meaningful `ParseException` containing the Line Number, Expected Element, and Found Element information so the user can easily identify the error.
Operator Precedence and Associativity:
The addition/subtraction (`expr`), multiplication/division (`term`), and factorization (`factor`) method hierarchy provides mathematical operator precedence (the priority of multiplication/division over addition) at the hardware level.
Left-associativity is implemented using `while(match(PLUS, MINUS))` loop structures, as proposed in Sebesta.

10. STATE DIAGRAMS

The Finite State Automata (FSA) used by the lexical analyzer to distinguish the most critical token categories are visualized below in digital format using Mermaid:
A. Identifier and Keyword Recognition State Diagram

```
[*] --> Başlangıç : Karakter Oku
   Başlangıç --> Hata : İlk Karakter Rakam/Özel Karakter
   Başlangıç --> HarfOkundu : [a-zA-Zçğışüöç̆ğ̇İŞÜÖ]
   HarfOkundu --> HarfOkundu : [a-zA-Z0-9_çğışüöç̆ğ̇İŞÜÖ]
   HarfOkundu --> TabloKontrol : Boşluk / Operatör / Noktalama (Tüketme)
   TabloKontrol --> KeywordBulundu : Sembol Tablosunda Mevcut?
   TabloKontrol --> IdentBulundu : Sembol Tablosunda Yok
   KeywordBulundu --> [*] : Token(KEYWORD) Üret
```

IdentBulundu --> [*] : Token(IDENTIFIER) Üret ve Sembol Tablosuna Ekle

B. State Diagram for Integer Literal Recognition

[*] --> Başlangıç : Karakter Oku
Başlangıç --> RakamOkundu : [0-9]
RakamOkundu --> RakamOkundu : [0-9]
RakamOkundu --> SayıTamamlandı : Rakam Dışı Karakter (Tüketme)
SayıTamamlandı --> [*] : Token(INT_LITERAL) Üret

C. Arithmetic and Relational Operators: Recognition State Diagram

stateDiagram-v2
[*] --> Başlangıç : Operatör Karakteri Oku
Başlangıç --> Artı : '+'
Artı --> Increment : '+' okunursa
Artı --> Plus : Rakam/Harf/Boşluk okunursa
Başlangıç --> Eşit : '='
Eşit --> Equals : '=' okunursa
Eşit --> Assign : Diğer durumlarda
Başlangıç --> Ünlem : '!'
Ünlem --> NotEquals : '=' okunursa
Ünlem --> LexHata : Başka karakter gelirse (Hata fırlat)

Increment --> [*] : Token(INCREMENT, "++")
Plus --> [*] : Token(PLUS, "+")
Equals --> [*] : Token(EQ, "==")
Assign --> [*] : Token(ASSIGN, "=")
NotEquals --> [*] : Token(NEQ, "!=")
LexHata --> [*] : LexerException Fırlat

11. SYMBOL TABLE / LOOKUP TABLE

The Symbol Table (`SymbolTable.java`) is a critical component that manages the uniqueness and roles of symbols defined in the program.

Symbol Table Architecture:

Separated Table Structure: The symbol table contains two distinct sub-maps within its internal structure:

1. `keywordTable (LinkedHashMap<String, TokenType>)`: A static keyword table that is loaded when the language is first initialized and cannot be modified.
2. `identifierTable (LinkedHashMap<String, TokenType>)`: A dynamic table that stores user-defined variable names discovered as the program runs.

Lookup: When a phrase is queried, it is first searched for in the `keywordTable`. If found there, it directly returns the keyword token type. If not found, the `identifierTable` is checked. If it is not found there either, it returns `null`, indicating that the word is appearing in the language for the first time.

Installation (Install): Newly discovered identifiers are added to the dynamic table using the `install(String name)` method. If the word to be added is a reserved keyword in the language (such as `if`, `variable`, etc.), the installation process is blocked to ensure lexical protection.

12.SAMPLE PROGRAMS

To test the project, four valid and four invalid test files covering all features of the language have been prepared.

A. VALID TEST FILES

1. Variable Declarations and Assignments (`tests/VariableDeclarationExample.txt`)

Demonstrates variable declarations, empty declarations, and dynamic assignments to variables.

```
// TürkDil — Değişken Bildirimi ve Aritmetik Örneği
degisken sayi = 42;
degisken toplam = 0;
degisken ad;
sayi = sayi + 10;
toplam = sayi * 2;
yazdir(toplam);
```

2. Arithmetic Expressions (`tests/ArithmeticExample.txt`)

Tests operator precedence, complex parentheses, and the unary minus operator.

```
// TürkDil — Aritmetik İfadeler Örneği
degisken a = 10;
degisken b = 3;
degisken sonuc;
sonuc = a + b;
yazdir(sonuc);
sonuc = a * b;
yazdir(sonuc);
sonuc = (a + b) * (a - b);
yazdir(sonuc);
sonuc = -a + b;
yazdir(sonuc);
```

3. Conditional Statements (`tests/ConditionalExample.txt`)

It tests the `eger ` and `yoksa` blocks, relational operators, and nested `if` structures.

```
// TürkDil — Koşullu İfadeler Örneği (eger / yoksa)
degisken x = 15;
degisken sonuc = 0;
eger (x > 10) {
   sonuc = 1;
   yazdir(sonuc);
}
eger (x == 15) {
   sonuc = sonuc + 100;
   yazdir(sonuc);
} yoksa {
```

```
   sonuc = 0;
   yazdir(sonuc);
}
degisken y = 5;
eger (y < 10) {
   eger (y > 0) {
      yazdir(y);
   } yoksa {
      yazdir(0);
   }
}
```

4. Loop Structures (`tests/LoopExample.txt`)

Tests the `dongu ` (while) and `icin` (for) loops, the increment operator (`++`), and nested loop structures.

TürkDil — Döngüler Örneği (dongu / icin)

```
degisken sayac = 0;
degisken toplam = 0;

sayac = 1;
dongu (sayac <= 5) {
   toplam = toplam + sayac;
   sayac++;
}
yazdir(toplam);

degisken i = 0;
icin (i = 0; i < 5; i++) {
   yazdir(i);
}
degisken dis = 0;
degisken ic = 0;
icin (dis = 0; dis < 3; dis++) {
   icin (ic = 0; ic < 3; ic++) {
      yazdir(ic);
   }
}
```

B.(INVALID TESTS FILES

1. Arithmetic and Character Error (`tests/InvalidArithmeticExample.txt`)

It contains invalid types not defined in the language (`ondlk`, `tam`), an invalid terminator (`:`), and undefined operators (`^`, `?=`).

```
ondlk pi = 3.14 :
tam yaricap = 5 :
tam alan = pi * yaricap ^ 2 :
alan ?= 50 :
```

2. Invalid Conditional Expression (`tests/InvalidConditionalExample.txt`)

It highlights syntax errors caused by the use of a non-existent word like `degilse ` and the use of `]]` as a block terminator.

```
tam puan = 85 :
eger (puan == 100) {
   yzdr "Harika" :
} degilse {
   yzdr "Daha cok calismalisin" :
]]
```

3. Invalid Loop Structure (`tests/InvalidLoopExample.txt`)

This is an error caused by writing the loop condition brackets in reverse order and using square brackets—which are not defined in the language—instead of curly braces.

```
tam sayac = 10 :
dongu ) sayac > 0 ( [[
   yzdr "Sayacin guncel degeri:
   sayac = sayac – 1
]]
```

4. Missing Terminator Error (`tests/InvalidVariableDeclarationExample.txt`)

This demonstrates that omitting the semicolon causes the parser to fail to distinguish between consecutive statements and throw an error.

```
tam yas = 20 :
tam yas = 25 :
isim = "Ahmet" :
cml sehir = "Ankara" ;
```

13. TEST RESULTS

A. ArithmeticExample.txt Test File Execution Output

```
==================================================
 SÖZCÜKSEL ANALİZ (LEXICAL ANALYSIS)
==================================================
Lexer başlatılıyor... [Sebesta Ch.4 lex() temelli]

Token Listesi:
┌──────────────────────────────────────────────────┐
|   1. [DEGISKEN     | "degisken"   | satir: 4]
|   2. [IDENTIFIER   | "a"          | satir: 4]
|   3. [ASSIGN       | "="          | satir: 4]
|   4. [INT_LITERAL  | "10"         | satir: 4]
|   5. [SEMICOLON    | ";"          | satir: 4]
|   6. [DEGISKEN     | "degisken"   | satir: 5]
|   7. [IDENTIFIER   | "b"          | satir: 5]
|   8. [ASSIGN       | "="          | satir: 5]
|   9. [INT_LITERAL  | "3"          | satir: 5]
|  10. [SEMICOLON    | ";"          | satir: 5]
|  11. [DEGISKEN     | "degisken"   | satir: 6]
|  12. [IDENTIFIER   | "sonuc"      | satir: 6]
```

| 13. [SEMICOLON | ";" | satir: 6]
| 14. [IDENTIFIER | "sonuc" | satir: 9]
| 15. [ASSIGN | "=" | satir: 9]
| 16. [IDENTIFIER | "a" | satir: 9]
| 17. [PLUS | "+" | satir: 9]
| 18. [IDENTIFIER | "b" | satir: 9]
| 19. [SEMICOLON | ";" | satir: 9]
| 20. [YAZDIR | "yazdir" | satir: 10]
| 21. [LPAREN | "(" | satir: 10]
| 22. [IDENTIFIER | "sonuc" | satir: 10]
| 23. [RPAREN | ")" | satir: 10]
| 24. [SEMICOLON | ";" | satir: 10]
| 25. [IDENTIFIER | "sonuc" | satir: 13]
| 26. [ASSIGN | "=" | satir: 13]
| 27. [IDENTIFIER | "a" | satir: 13]
| 28. [MULT | "*" | satir: 13]
| 29. [IDENTIFIER | "b" | satir: 13]
| 30. [SEMICOLON | ";" | satir: 13]
| 31. [YAZDIR | "yazdir" | satir: 14]
| 32. [LPAREN | "(" | satir: 14]
| 33. [IDENTIFIER | "sonuc" | satir: 14]
| 34. [RPAREN | ")" | satir: 14]
| 35. [SEMICOLON | ";" | satir: 14]
| 36. [IDENTIFIER | "sonuc" | satir: 17]
| 37. [ASSIGN | "=" | satir: 17]
| 38. [LPAREN | "(" | satir: 17]
| 39. [IDENTIFIER | "a" | satir: 17]
| 40. [PLUS | "+" | satir: 17]
| 41. [IDENTIFIER | "b" | satir: 17]
| 42. [RPAREN | ")" | satir: 17]
| 43. [MULT | "*" | satir: 17]
| 44. [LPAREN | "(" | satir: 17]
| 45. [IDENTIFIER | "a" | satir: 17]
| 46. [MINUS | "-" | satir: 17]
| 47. [IDENTIFIER | "b" | satir: 17]
| 48. [RPAREN | ")" | satir: 17]
| 49. [SEMICOLON | ";" | satir: 17]
| 50. [YAZDIR | "yazdir" | satir: 18]
| 51. [LPAREN | "(" | satir: 18]
| 52. [IDENTIFIER | "sonuc" | satir: 18]
| 53. [RPAREN | ")" | satir: 18]
| 54. [SEMICOLON | ";" | satir: 18]
| 55. [IDENTIFIER | "sonuc" | satir: 21]
| 56. [ASSIGN | "=" | satir: 21]
| 57. [MINUS | "-" | satir: 21]

```
|   58. [IDENTIFIER    | "a"          | satir: 21]
|   59. [PLUS          | "+"          | satir: 21]
|   60. [IDENTIFIER    | "b"          | satir: 21]
|   61. [SEMICOLON     | ";"          | satir: 21]
|   62. [YAZDIR        | "yazdir"     | satir: 22]
|   63. [LPAREN        | "("          | satir: 22]
|   64. [IDENTIFIER    | "sonuc"      | satir: 22]
|   65. [RPAREN        | ")"          | satir: 22]
|   66. [SEMICOLON     | ";"          | satir: 22]
|   67. [EOF           | "EOF"        | satir: 23]
└──────────────────────────────────────────────────┘
Toplam 67 token üretildi.
==================================================
  SEMBOL TABLOSU (SYMBOL TABLE)
==================================================
╔══════════════════════════════════════════════════╗
║       SEMBOL TABLOSU / SYMBOL TABLE    ║
╠══════════════════════════════════════════════════╣
║  [ANAHTAR SÖZCÜKLER / KEYWORDS]          ║
╠══════════════════════════════════════════════════╣
║  degisken          → DEGISKEN         ║
║  eger              → EGER             ║
║  yoksa             → YOKSA            ║
║  dongu             → DONGU            ║
║  icin              → ICIN             ║
║  yazdir            → YAZDIR           ║
║  dogru             → DOGRU            ║
║  yanlis            → YANLIS           ║
║  ve                → VE               ║
║  veya              → VEYA             ║
║  degil             → DEGIL            ║
╠══════════════════════════════════════════════════╣
║  [TANIMLAYICILAR / IDENTIFIERS]          ║
╠══════════════════════════════════════════════════╣
║  a                 → IDENTIFIER       ║
║  b                 → IDENTIFIER       ║
║  sonuc             → IDENTIFIER       ║
╚══════════════════════════════════════════════════╝

==================================================
  SÖZDİZİMİ ANALİZİ (SYNTAX ANALYSIS)
==================================================
Parser başlatılıyor... [Sebesta Ch.4 Recursive-Descent temelli]
╔══════════════════════════════════════════════════╗
║        SÖZDİZİMİ ANALİZİ BAŞLADI                ║
[program] Ayrıştırma başlıyor...
```

[bildirim] Değişken: a
[bildirim] Başlangıç değeri atanıyor...
  [factor] Tam sayı sabiti: 10
[bildirim] ✓ Tamamlandı: a
[bildirim] Değişken: b
[bildirim] Başlangıç değeri atanıyor...
  [factor] Tam sayı sabiti: 3
[bildirim] ✓ Tamamlandı: b
[bildirim] Değişken: sonuc
[bildirim] ✓ Tamamlandı: sonuc
[atama] Sağ taraf hesaplanıyor: sonuc = ...
  [factor] Tanımlayıcı: a
  [expr] Operatör: +
  [factor] Tanımlayıcı: b
[atama] ✓ Tamamlandı: sonuc
[yazdir] Çıktı deyimi işleniyor...
  [factor] Tanımlayıcı: sonuc
[yazdir] ✓ Çıktı deyimi tamamlandı.
[atama] Sağ taraf hesaplanıyor: sonuc = ...
  [factor] Tanımlayıcı: a
  [term] Operatör: *
  [factor] Tanımlayıcı: b
[atama] ✓ Tamamlandı: sonuc
[yazdir] Çıktı deyimi işleniyor...
  [factor] Tanımlayıcı: sonuc
[yazdir] ✓ Çıktı deyimi tamamlandı.
[atama] Sağ taraf hesaplanıyor: sonuc = ...
  [factor] Parantezli ifade başlıyor...
  [factor] Tanımlayıcı: a
  [expr] Operatör: +
  [factor] Tanımlayıcı: b
  [factor] Parantezli ifade tamamlandı.
  [term] Operatör: *
  [factor] Parantezli ifade başlıyor...
  [factor] Tanımlayıcı: a
  [expr] Operatör: -
  [factor] Tanımlayıcı: b
  [factor] Parantezli ifade tamamlandı.
[atama] ✓ Tamamlandı: sonuc
[yazdir] Çıktı deyimi işleniyor...
  [factor] Tanımlayıcı: sonuc
[yazdir] ✓ Çıktı deyimi tamamlandı.
[atama] Sağ taraf hesaplanıyor: sonuc = ...
  [factor] Tekli eksi (unary minus)

[factor] Tanımlayıcı: a
[expr] Operatör: +
[factor] Tanımlayıcı: b
[atama] ✓ Tamamlandı: sonuc
[yazdir] Çıktı deyimi işleniyor...
[factor] Tanımlayıcı: sonuc
[yazdir] ✓ Çıktı deyimi tamamlandı.
[program] Tüm deyimler başarıyla ayrıştırıldı.
╟ ✓ BAŞARILI: Program sözdizimi geçerlidir. ╢
==========
ÖZET
==========
✓ Sözcüksel Analiz : BAŞARILI (67 token)
✓ Sözdizimi Analizi: BAŞARILI
✓ Sembol Tablosu : 3 tanımlayıcı kayıtlı

B. InvalidArithmeticExample.txt Error Analysis Output

When this test file is run, the error log generated by the system due to characters for which there are no language equivalents and syntactically incorrect expressions is as follows:

==========

SÖZCÜKSEL ANALİZ (LEXICAL ANALYSIS)

==========

Lexer başlatılıyor... [Sebesta Ch.4 lex() temelli]

[SÖZCÜKSEL HATA] Satir 1: Tanimsiz karakter '.' (Unicode: U+002E)

[SÖZCÜKSEL HATA] Satir 4: Tanimsiz karakter '^' (Unicode: U+005E)

[SÖZCÜKSEL HATA] Satir 5: Tanimsiz karakter '?' (Unicode: U+003F)

Token Listesi:

┌──────────┐

│ 1. [IDENTIFIER | "ondlk" | satir: 1]

│ 2. [IDENTIFIER | "pi" | satir: 1]

│ 3. [ASSIGN | "=" | satir: 1]

│ 4. [INT_LITERAL | "3" | satir: 1]

│ 5. [UNKNOWN | "." | satir: 1]

│ 6. [INT_LITERAL | "14" | satir: 1]

│ 7. [UNKNOWN | ":" | satir: 1]

...

└──┘

==========

SÖZDİZİMİ ANALİZİ (SYNTAX ANALYSIS)

==========

Parser başlatılıyor... [Sebesta Ch.4 Recursive-Descent temelli]

╟ SÖZDİZİMİ ANALİZİ BAŞLADI ╢

[program] Ayrıştırma başlıyor...

❌ **SÖZDİZİMİ HATASI: Satir 1: '=' (atama operatörü) beklendi, ancak 'pi' (IDENTIFIER) bulundu.**

```> [!WARNING]
Error Reporting Mechanism:
As can be seen, since there is no keyword named `ondlk` defined in the language, the lexer interpreted it as an identifier (`IDENTIFIER`). The following word `pi` is also an identifier. According to the parser rules, an expression beginning with an identifier must be an assignment statement (`assignment`) and must be immediately followed by the `=` operator. However, when the parser encountered the `pi` (IDENTIFIER) token, it detected that the assignment operator was missing and safely terminated execution by reporting the error: “Line 1: ‘=’ expected, but ‘pi’ (IDENTIFIER) found.”

14. CONCLUSION

This project successfully translates the theoretical compiler stages presented in Chapter 4 of Robert W. Sebesta’s *Concepts of Programming Languages* into a practical computer software application. The designed programming language and its compiler front-end (Lexer, Parser, and Symbol Table) implementation have yielded the following outcomes:

1. The Relationship Between Theory and Practice: Students gained practical experience in how regular expressions and finite-state automata (FSA) are applied in lexical analysis, and how EBNF rules and operator precedence hierarchies are implemented within recursive-descent parser methods.

2. The Importance of Error Handling: It was recognized that in cases where a programmer writes erroneous code, it is critically important for the compiler to generate user-friendly error messages specifying exactly which line the error occurred on, the cause of the error, and the token mismatch—rather than simply reporting an error and terminating.

3. Turkish Character Integration: Unlike standard ASCII-based compilers, the ability to seamlessly process character sets specific to the language’s culture through Unicode-based character scans, as well as the feasibility of applying Turkish programming concepts, has been verified.

In conclusion, all of the minimum requirements defined at the start of the project (variable declarations, assignments, arithmetic expressions, ` eger-yoksa ` conditional blocks, `dongu` and `icin` loops, Turkish character support, a symbol table, and error-handling mechanisms) have been fully implemented and tested in a stable and consistent manner. This work has laid a very solid foundation for future optimizations in the language (such as Intermediate Code Generation or direct Interpreter execution phases).